



TRABAJO PRACTICO INTEGRADOR - PROGRAMACIÓN 2

Tecnicatura Universitaria en Programacion

Proyecto: Food Store - Sistema de Gestión de Pedidos de Comida

Materia: Programación 2

Alumno: Centurión Bartlett Juan Rodolfo

Fecha de Entrega: 22/6

Índice

1. Marco Teórico	3
2. Decisiones de Diseño y Arquitectura	3
3. Estructura del Modelo de Datos (UML)	4
4. Implementación de Reglas de Negocio y Excepciones	4
5. Dificultades Encontradas y Soluciones	5
6. Bibliografía y Webgrafía	5
7. Enlaces del Proyecto	5

1) Marco Teórico

El desarrollo de este sistema se fundamenta en las especificaciones de **Java 21** y los pilares de la **Programación Orientada a Objetos (POO)**. Al tratarse de una solución modular que prescinde temporalmente de una base de datos relacional persistente, el núcleo del almacenamiento operativo se apoya firmemente en el **Framework de Colecciones de Java**.

Se optó por el uso de `ArrayList` para la gestión en memoria de todas las entidades debido a su eficiencia en operaciones de lectura y recorrido secuencial mediante iteradores. La estructuración del código sigue un enfoque de diseño limpio y fuertemente tipado:

- **Abstracción y Herencia:** Permite reutilizar atributos comunes (como identificadores y banderas de estado) reduciendo la redundancia de código.
- **Encapsulamiento:** Protege el estado interno de los objetos mediante modificadores de acceso y métodos de acceso controlados.
- **Polimorfismo e Interfaces:** Define contratos de comportamiento obligatorios que independizan la lógica de negocio de la estructura de datos.

2) Decisiones De Diseño y Arquitectura

El sistema **Food Store** se estructuró utilizando una arquitectura desacoplada en capas bien definidas dentro del paquete `integrado.prog2`, garantizando la separación de responsabilidades:

1. **Capa de Modelo (`integrado.prog2.entities y enums`):** Contiene las clases que representan el negocio. Se aplicó herencia creando una clase abstracta `Base` para unificar el atributo `id` y el estado lógico eliminado que comparten todas las entidades principales (`Categoria`, `Producto`, `Usuario`, `Pedido`).
2. **Capa de Servicios (`integrado.prog2.services`):** Centraliza la lógica de negocio, las validaciones y las colecciones en memoria (`List<T>`). Las clases de servicio (`ProductoService`, `UsuarioService`, etc.) actúan como intermediarias aisladas, impidiendo que la interfaz de usuario manipule los datos directamente.
3. **Capa de Controladores/Vista (`integrado.prog2.Main`):** Gestiona la interacción con el operador mediante consolas de comandos limpias. Implementa submenús iterativos basados en estructuras `switch-case` y capturas de flujo mediante bloques `try-catch`.

3) Estructura del modelo de datos

La consistencia y las relaciones del dominio se mapearon directamente respetando las consignas obligatorias del enunciado:

- **Relación 1 a 1 Unidireccional Efectiva:** Cada instancia de Producto conoce e incluye una referencia directa a la instancia de la Categoría a la que pertenece.
- **Relación Maestro-Detalle:** La entidad Pedido compone de forma interna una colección de DetallePedido. Un detalle no puede existir huérfano fuera de un pedido.
- **Interfaz Calculable:** Implementada directamente en la entidad Pedido, forzando el uso del método `calcularTotal()` para consolidar la suma de los subtotales (`cantidad * precio`) calculados dinámicamente en los detalles del pedido.

4) Implementación de reglas de negocio y excepciones

Para asegurar que el sistema no sufra inconsistencias ni se cierre inesperadamente ante entradas erróneas del usuario, se diseñó un esquema robusto de validaciones asistido por excepciones personalizadas:

- **ReglaNegocioException:** Se lanza cuando una operación vulnera los requisitos lógicos expuestos en las historias de usuario. Por ejemplo: intentar registrar un usuario con un correo electrónico que ya existe y está activo en el sistema, o intentar vender un producto con stock insuficiente.
- **EntidadNoEncontradaException:** Se dispara de manera controlada al realizar búsquedas por `id` sobre objetos que no existen en las listas o que han sido dados de baja de manera lógica.
- **Control de Transacciones de Stock:** Al registrar un pedido, la capa `PedidoService` descuenta el stock de manera preventiva. Si ocurre algún error inesperado en la adición de los detalles, un mecanismo de reversión intercepta la excepción, devuelve el stock correspondiente a los productos y cancela la creación en memoria del pedido.

5) Dificultades encontradas y soluciones

1. **Dificultad (Codificación de Consola):** Inicialmente, la salida de consola de NetBeans mostraba caracteres extraños (rombos con signos de pregunta) al imprimir textos con tildes o caracteres especiales, afectando la prolijidad visual del menú.
 - a. *Solución:* Se estandarizaron todas las cadenas de texto del menú y de los mensajes de error removiendo tildes y caracteres especiales directamente en las instrucciones `System.out.println`, garantizando una compatibilidad perfecta en cualquier terminal de ejecución.
2. **Dificultad (Inconsistencia de Datos en Cancelaciones):** Un desafío crítico consistía en evitar que los productos perdieran unidades de stock si la carga de un pedido se cancelaba a mitad de camino por falta de stock en un segundo artículo.
 - a. *Solución:* Se desarrolló el método `cancelarCreacionPedido(Pedido pedido)` en el servicio. Este método realiza un ciclo inverso iterando sobre los detalles cargados hasta el momento del fallo para restaurar exactamente las cantidades al stock de cada producto involucrado antes de descartar el pedido.

6) Bibliografía

- Oracle Corporation. (2023). *Java Platform, Standard Edition Internationalization Guide (Release 21)*. Recuperado de <https://docs.oracle.com/en/java/javase/21/>
- Bloch, J. (2018). *Effective Java* (3rd ed.). Addison-Wesley Professional.
- Universidad Tecnológica Nacional. *Material de lectura cátedra Programación 2 (Paradigma Orientado a Objetos y Colecciones)*.

7) Enlaces

- **Video Demostrativo (YouTube):** <https://youtu.be/5glssMYKpPg>
- **Repositorio de Código (GitHub):** <https://github.com/CenturionJuan/UTN-TUPaD-TPI-Programacion2/tree/main>